

Diagnostic tests

David Moreno

Sensitivity and Specificity

Test:

Sensitivity is the probability of a test to detect disease in sick patients.

Specificity is the probability of a test to not detect disease in healthy people.

Results:

Positive predictive value is the probability that a person has the disease if the test result is positive.

Negative predictive value is the probability that a person is healthy if the test result is negative.

How to quickly assess sensitivity and specificity?

```
# Install the caret package and load the data
# install.packages("caret")
# Be sure to have the reference and the data with same levels
library(caret)
library(readxl)

protein_data <- read_excel("/Users/dfmoreno/Dropbox/Lectures/data/protein_data.xlsx")
protein_data$Test <- as.factor(protein_data$Test)
levels(protein_data$Test) # check your categories

protein_data$Gold_Standard <- as.factor(protein_data$Gold_Standard)
levels(protein_data$Gold_Standard) # check your categories

confusionMatrix(protein_data$Test, # the test you are running
                 protein_data$Gold_Standard, # the reference
                 positive = "Malignant") # indicate the positive result
```

Interpret the results of the confusion matrix. See more details [here](#)

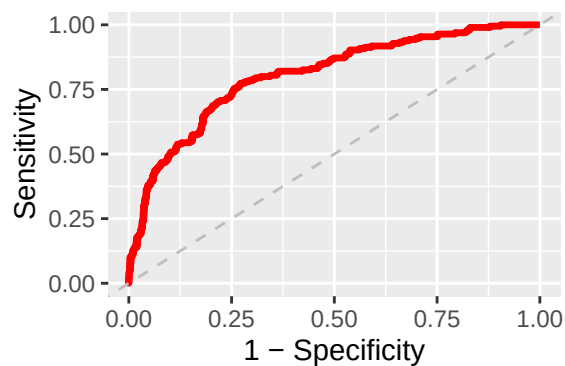
Now you can calculate an AUC and plot the curve

```
# Install the pROC package
# install.packages("pROC")
library(pROC)
roc_obj <- roc(response = protein_data$Gold_Standard,
               predictor = protein_data$Protein_Expression)

auc(roc_obj)
# plot(roc_obj)

roc_df <- data.frame(
  specificity = roc_obj$specificities,
  sensitivity = roc_obj$sensitivities
)

ggplot(roc_df, aes(x = 1 - specificity, y = sensitivity)) +
  geom_line(color = "red", size = 1.2) +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed", color = "grey") +
  xlab("1 - Specificity") +
  ylab("Sensitivity")
```



Predict a diagnosis

The data used in this part is from 1995 (many things have changed). But you can change the variables to other parameters that you use now in the diagnostic setting.

```

# install.packages("mlbench")
# install.packages("randomForest")
library(mlbench)
library(caret)
library(randomForest)

data("BreastCancer") # from mlbench

# Cl.thickness    Clump Thickness (1-10)
# Cell.size       Uniformity of Cell Size (1-10)
# Cell.shape      Uniformity of Cell Shape (1-10)
# Marg.adhesion   Marginal Adhesion (1-10)
# Epith.c.size    Single Epithelial Cell Size (1-10)
# Bare.nuclei     Bare Nuclei (1-10)
# Bl.cromatin     Bland Chromatin (1-10)
# Normal.nucleoli Normal Nucleoli (1-10)
# Mitoses         Mitoses (1-10)

# First, remove observations with missing values with "na.omit"
bc_data <- na.omit(BreastCancer)

# Remove the Id (no need)
bc_data$Id <- NULL

# Convert to factor
bc_data$Class <- factor(bc_data$Class, levels = c("benign", "malignant"))

# Divide you original data into training and validation/test
set.seed(123) # makes reproducible results when using random algorithms
trainIndex <- createDataPartition(bc_data$Class, p = 0.7, list = FALSE)
trainData <- bc_data[trainIndex, ]
testData  <- bc_data[-trainIndex, ]

# Train your model
# rpart is a decision tree algorithm
# There are multiple methods including glm, lm, neural networks...
model <- train(Class ~ ., data = trainData, method = "rpart")

# Predictions
predictions <- predict(model, newdata = testData)

# Confusion matrix of your predictions

```

```
result <- confusionMatrix(predictions, testData$Class, positive = "malignant")
result
```

Confusion Matrix and Statistics

	Reference	
Prediction	benign	malignant
benign	130	11
malignant	3	60

Accuracy : 0.9314
 95% CI : (0.8875, 0.962)
 No Information Rate : 0.652
 P-Value [Acc > NIR] : < 2e-16

 Kappa : 0.8447

 McNemar's Test P-Value : 0.06137

 Sensitivity : 0.8451
 Specificity : 0.9774
 Pos Pred Value : 0.9524
 Neg Pred Value : 0.9220
 Prevalence : 0.3480
 Detection Rate : 0.2941
 Detection Prevalence : 0.3088
 Balanced Accuracy : 0.9113

 'Positive' Class : malignant

Inter-rater reliability

Sometimes you want to compare two raters or two diagnostic tests and see whether they agree and how much.

```
# Install the irr package and download the diagnoses data
# install.packages("irr")
library(irr)
library(tidyverse)
```

```
data("diagnoses")
str(diagnoses)

# Calculate the % of agreement between raters
agree(diagnoses)
```

In case of multiple raters, a high agreement has a lot of value. But between two raters there is a chance that they agree by chance, even in the case that the tests were performed independently. Cohen's kappa is a calculation that adjust the agreement by chance.

```
# kappa <0.2: poor
# kappa 0.21-0.40: low
# kappa 0.41-0.60: moderate
# kappa 0.61-0.80: good
# kappa 0.81-1.00: excelent

# install.packages('vcd')
library(vcd)
# If you only have two raters
# kappa2(diagnoses) it fails

# More than 2 raters
kappam.fleiss(diagnoses)

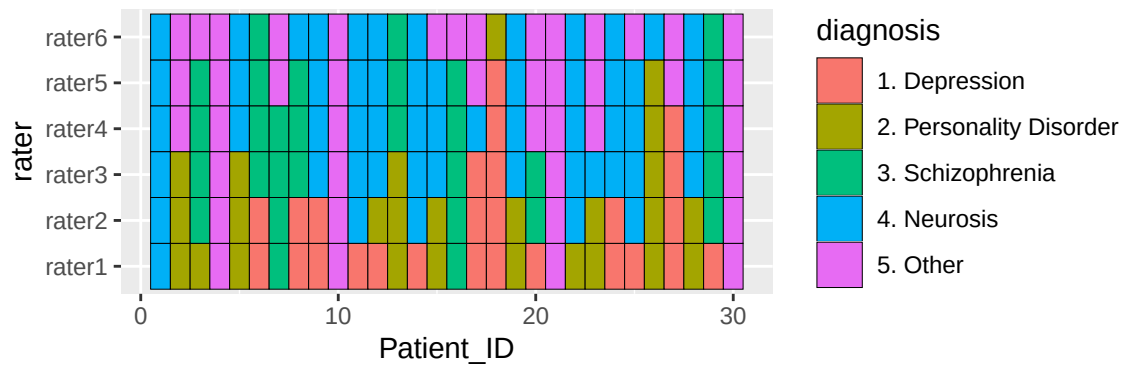
# Make a subset of diagnoses
diagnoses_2 <- diagnoses %>% select(rater1, rater2)
# diagnoses_3 <- subset(diagnoses, select=c("rater1", "rater2")) # another way
kappa2(diagnoses_2)

# Prepare to plot
table <- table(diagnoses_2$rater1, diagnoses_2$rater2)
# agreementplot(table)

# Use the full dataset
diagnoses <- diagnoses %>% mutate(Patient_ID = 1:nrow(.))

# Transformation for plotting
diagnoses_long <- diagnoses %>%
  pivot_longer(cols = starts_with("rater"),
               names_to = "rater",
               values_to = "diagnosis"
  )
```

```
# Actually plotting
ggplot(diagnoses_long, aes(x = Patient_ID, y = rater, fill = diagnosis)) +
  geom_tile(color = "black")
```



This work is licensed under a
[Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

